NIMBUS — Design Lock

10 Oct 2026 · Derrick, Vishwa Thangapandiyan

Locked scope

NIMBUS predicts which Chennai streets will flood in the next few hours, routes people around them, and gives ward officers a ranked action list built by a team of agents. Track: **Heat and Water** (floods, monsoon waterlogging).

The problem in one line: rain alerts are city-wide, flooding is street-level, and the response starts only after streets are already under water.

Positioning for judges: not another flood map. It is a decision system that learns from every citizen report and turns predictions into approved actions.

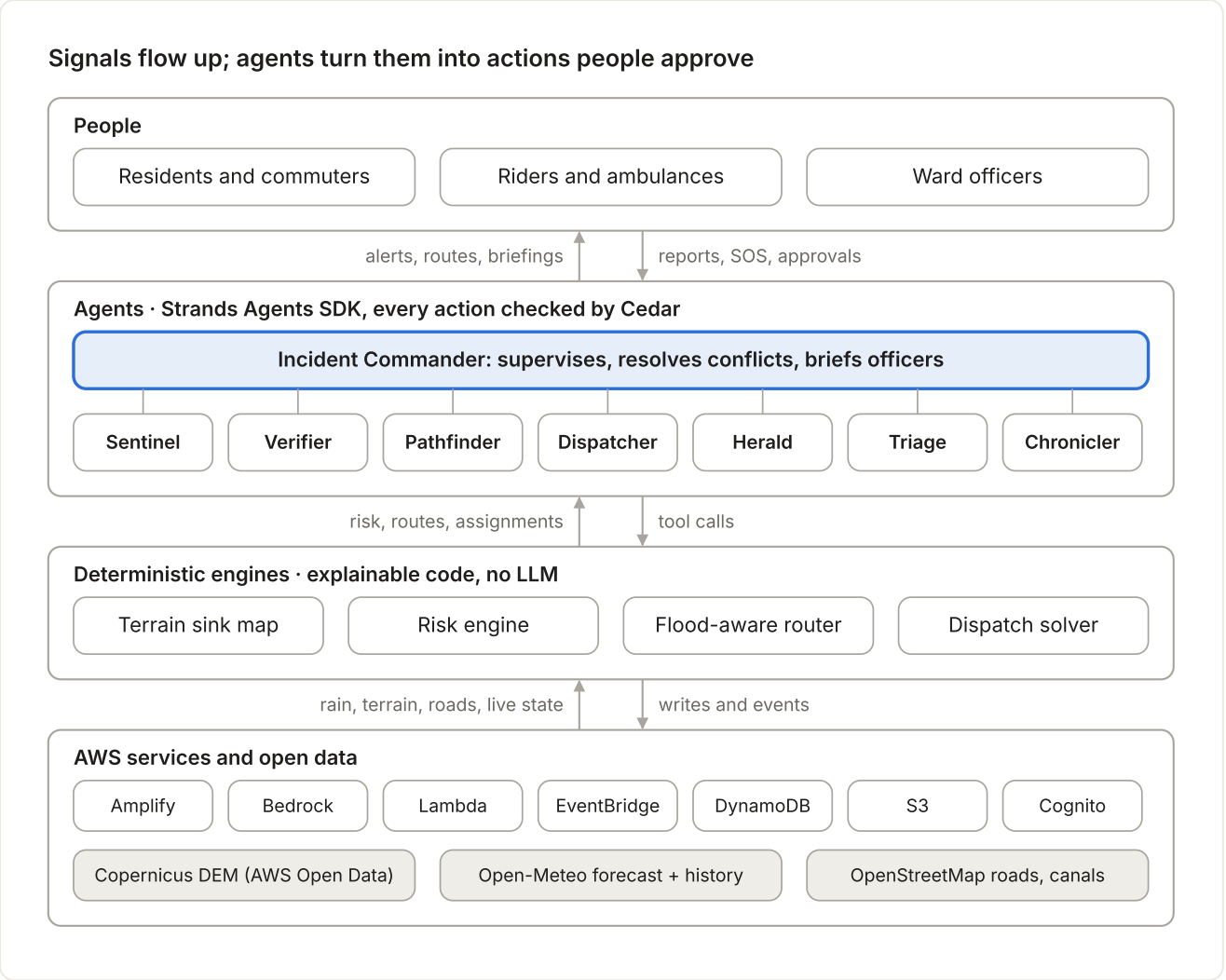
User	What they need	What NIMBUS gives them
Resident or commuter	Will my street flood? Should I move my vehicle?	Street-level risk and alerts for saved places
Two-wheeler rider or ambulance	A route that won't stall in water	Routes that avoid flooded streets, per vehicle type
Ward officer	Where to send pumps and crews first	A ranked dispatch list, hazard queue and briefings
Planner or commissioner	Which drains keep failing	Per-street flood history and after-action reports

In scope for the demo: one or two wards (Velachery, optionally Pallikaranai), English and Tamil, a web app plus a Telegram bot, live mode and Storm Replay mode.

Out of scope this weekend: SMS (needs DLT registration), the real storm-drain network, citywide coverage, hardware water-level sensors.

System at a glance

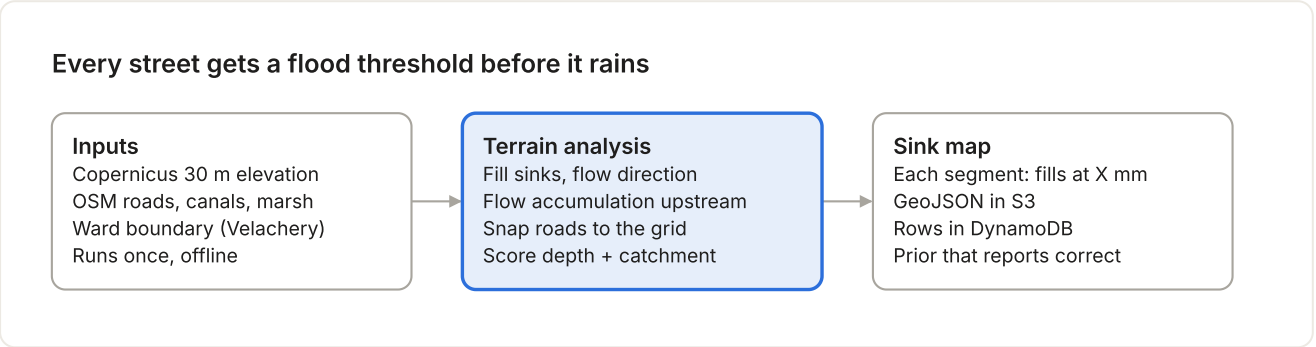
Four layers: people at the top, a team of agents that decides, deterministic engines that calculate, and AWS plus open data underneath.



The split matters for judging: the math (risk, routing, assignment) is plain, explainable code, and the agents handle judgement, verification and communication on top of it.

F1 · Terrain sink map

Every street segment gets a "fills at X mm/hr" threshold before any rain falls. This is the static foundation the rest of the system builds on.



1. Clip the Copernicus GLO-30 elevation model (AWS Open Data registry) to the ward's bounding box.
2. Fill depressions, then compute flow direction and flow accumulation with richdem, pysheds or WhiteboxTools.

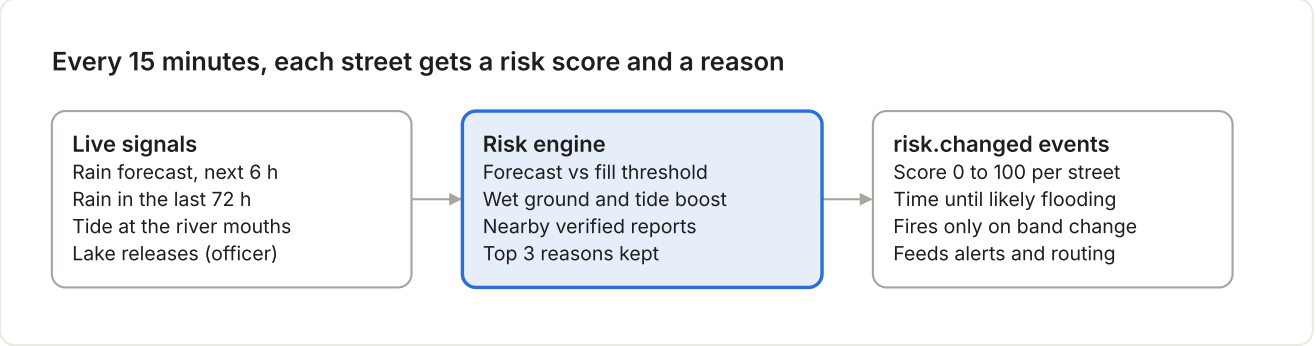
3. Pull roads, canals and the marsh edge from OpenStreetMap through the Overpass API, and snap road segments to the grid.
4. Score each segment on depression depth, upstream catchment area and distance to the nearest canal or river.
5. Write the result as GeoJSON to S3 and one row per segment to DynamoDB.

Honest limit, say it in the video: 30 m elevation and no drain network make this a prior, not the truth. F3's citizen reports correct it street by street.

Priority: must ship. It runs once, offline, as a script.

F2 · Live risk engine

Every 15 minutes, each street gets a risk score from 0 to 100, an estimated time until it floods, and the top three reasons why.



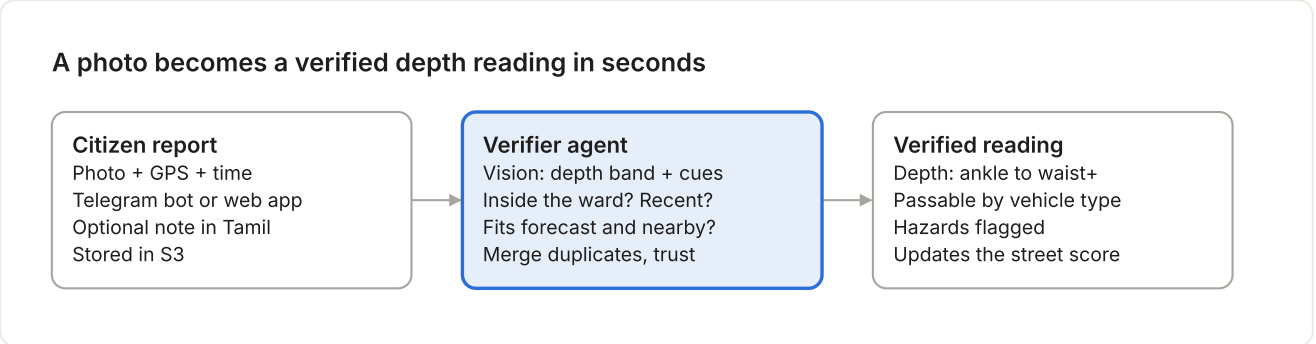
- **Forecast rain:** hourly precipitation for the next few hours from Open-Meteo, compared with each segment's fill threshold from F1.
- **Antecedent rain (new):** rain over the past 72 hours. Saturated ground floods faster, which is why the second storm in a week hits harder.
- **Tide (new):** high tide slows drain outflow at the Adyar and Cooum mouths. Use a precomputed tide table for the weekend.
- **Lake releases (new):** upstream reservoir releases feed the rivers. For the demo, the officer enters these by hand.
- **Verified reports:** a nearby verified flood reading raises the score of connected low segments.

The scoring is deterministic code, not an LLM, so every number can be explained. EventBridge Scheduler triggers a Lambda that writes scores to DynamoDB and publishes `risk.changed` only when a segment crosses a band (low, watch, likely, flooded). That keeps the agents from reacting to noise.

Priority: must ship.

F3 · Citizen reports with depth from photo

A resident sends a photo; within seconds it becomes a verified depth reading on the map, and the street's threshold learns from it. This is the feature that makes the map get smarter over time.

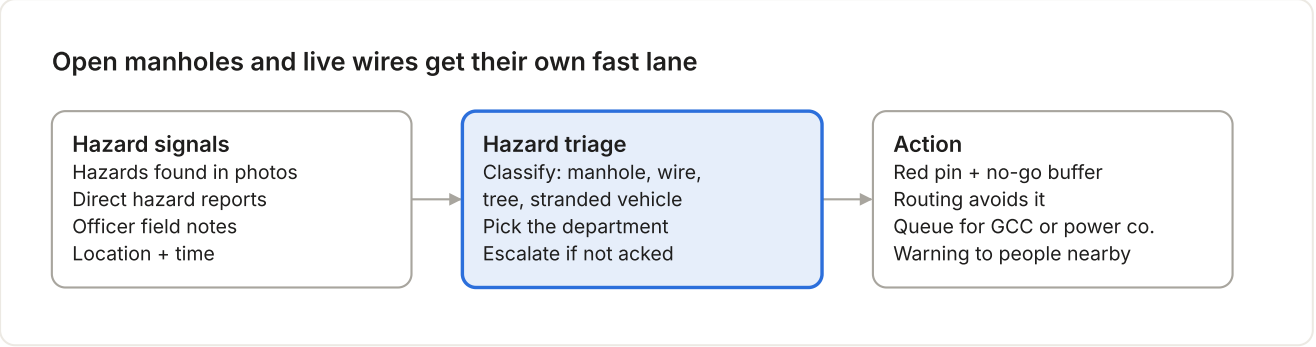


- **Vision step:** a multimodal model on Bedrock returns structured JSON: depth band (dry, ankle, knee, waist, above waist), the reference object it used (kerb, tyre, knee, wall), confidence, hazards seen, and whether a two-wheeler, car or ambulance could pass.
- **Checks:** GPS inside the ward, photo time recent (EXIF where present), consistent with the forecast and with neighbouring reports. Reports within about 50 m and 20 minutes are merged.
- **Trust score (new):** reporters whose readings keep matching others earn more weight; outliers get flagged, not deleted.
- **Learning map (new):** each verified reading nudges that segment's fill threshold, so the 30 m terrain prior gets corrected street by street.
- **Privacy (stretch):** blur faces and number plates before storing.

Priority: must ship. This is your strongest computer-vision moment in the video.

F4 · Hazard layer (new)

Open drains, missing manhole covers and live wires in standing water are among the deadliest monsoon dangers, and they are invisible under brown water. They get their own fast lane, separate from depth.

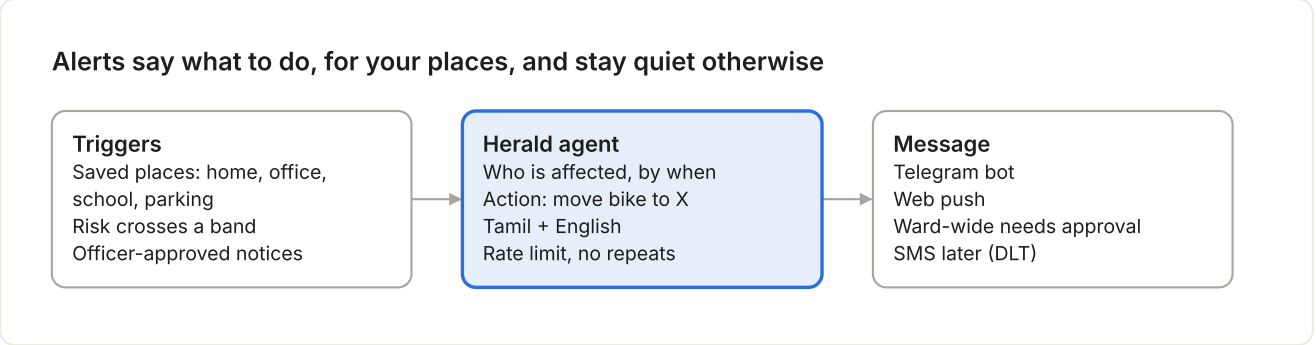


- **Detection is nearly free:** the F3 vision prompt already returns hazards, so this feature mostly adds routing and escalation.
- **Classes:** open or missing manhole cover, fallen power line or flooded junction box, fallen tree, stranded vehicle.
- **Routing to the right owner:** manholes and trees go to the corporation's queue, electrical hazards to the power distribution company's queue.
- **Escalation:** unacknowledged hazards escalate to the Incident Commander and get repeated warnings to people nearby.
- **Routing tie-in:** every hazard adds a no-go buffer that F6 avoids.

Priority: detection and the map pin are must ship; department queues and escalation are should.

F5 · Smart alerts

Alerts are tied to the places a person cares about, tell them exactly what to do, arrive in Tamil or English, and stay quiet when nothing changes.

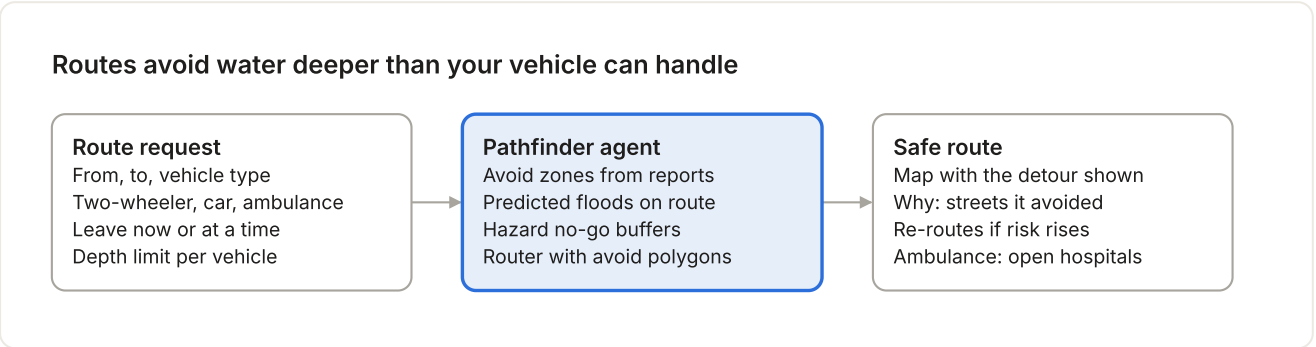


- **Saved places:** home, office, a child's school, where the bike is parked. Each is a small geofence over street segments.
- **Action-first wording:** "Your street is likely waterlogged in about 2 hours. Move your two-wheeler to the higher ground at X now." Higher ground comes from the F1 terrain data, so the suggestion is concrete.
- **Vehicle rescue nudge (new):** flooded parked two-wheelers and cars are one of the most common monsoon losses. A move-your-vehicle alert is cheap and very relatable in the video.
- **Anti-fatigue rules:** alert only when the band goes up, cap alerts per place per few hours, and send an all-clear when it drops.
- **Channels:** Telegram bot and web push. SMS stays out until DLT registration is done.

Priority: must ship (Telegram plus saved places).

F6 · Flood-aware safe routing

Routes avoid water deeper than the chosen vehicle can handle, including streets that are predicted to flood before you reach them.



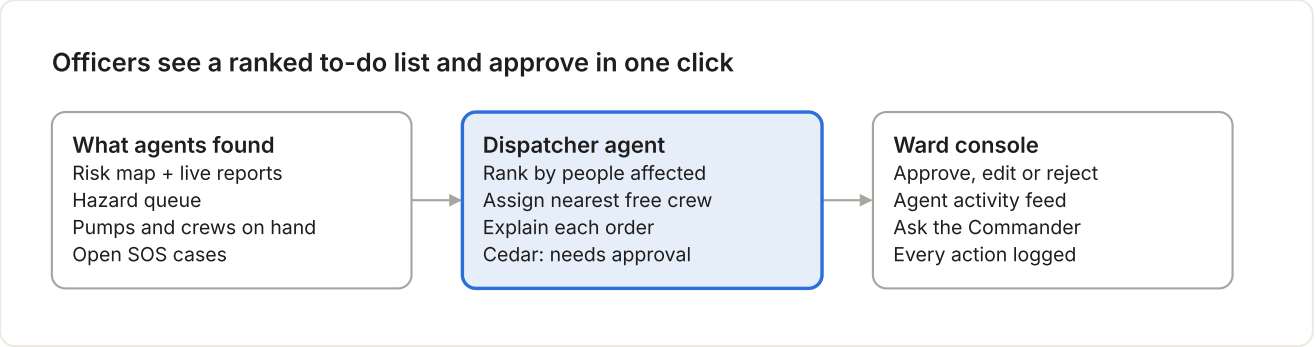
- **Vehicle profiles:** two-wheeler, car and ambulance each have a depth limit. Keep the limits configurable rather than hard-coding numbers you can't defend.
- **Avoid zones:** verified depth readings above the vehicle's limit, hazard buffers from F4, and segments in the "likely" band whose time-to-flood is shorter than your arrival time there (new, time-aware).
- **Router:** OpenRouteService directions accept avoid polygons, which makes it the safe default. If you want more AWS surface, check whether Amazon Location Service route avoidance fits your case.

- **Explanation:** the Pathfinder agent writes one line on why the detour exists, such as "avoids 2 streets reported knee-deep."
- **Ambulance mode (stretch):** route to the nearest hospital whose access road is passable.

Priority: must ship for two-wheeler and ambulance profiles.

F7 · Ward command console and dispatch

The ward officer opens one screen and sees a ranked to-do list the agents built, each item with its reason, and approves it in one click.

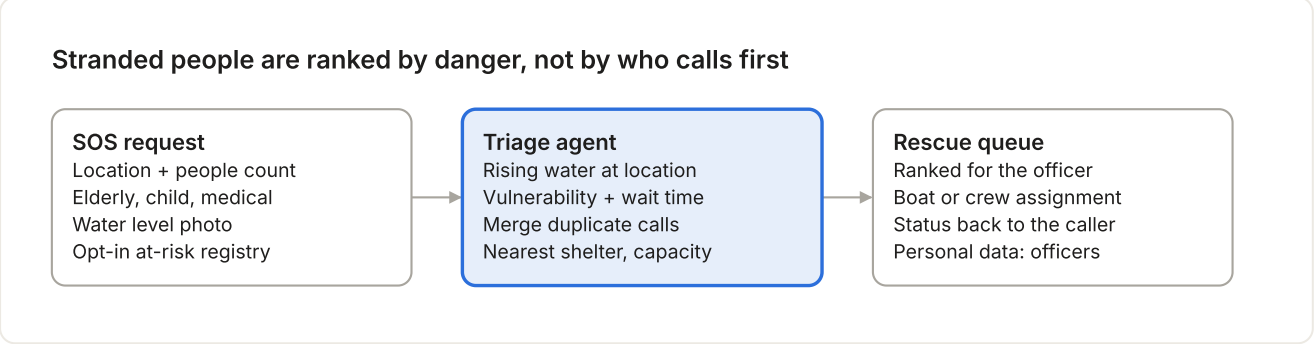


- **Live map:** risk shading, verified reports, hazard pins, crew positions.
- **Dispatch queue:** "Send pump 2 to Velachery Main Road, 3 verified knee-deep reports, about 400 buildings in the catchment." Building counts come from OpenStreetMap as an impact proxy.
- **Resource board:** pumps, desilting crews and tree crews, with status (free, en route, working).
- **Assignment:** a greedy solver ranks by impact over travel time; OR-Tools is the upgrade if time allows. The agent explains, the solver decides.
- **Agent activity feed (new):** a live stream of what each agent saw, decided and why. It is the most convincing multi-agent visual for the video.
- **Ask the Commander:** a chat box for questions like "what changed in the last hour?"
- **Access:** officers sign in with Cognito; every approval, edit and rejection is logged.

Priority: must ship (map, queue, approve, activity feed).

F8 · SOS triage and vulnerable residents (new)

When people are stranded, requests are ranked by danger: how fast water is rising where they are, who is there, and how long they have waited.

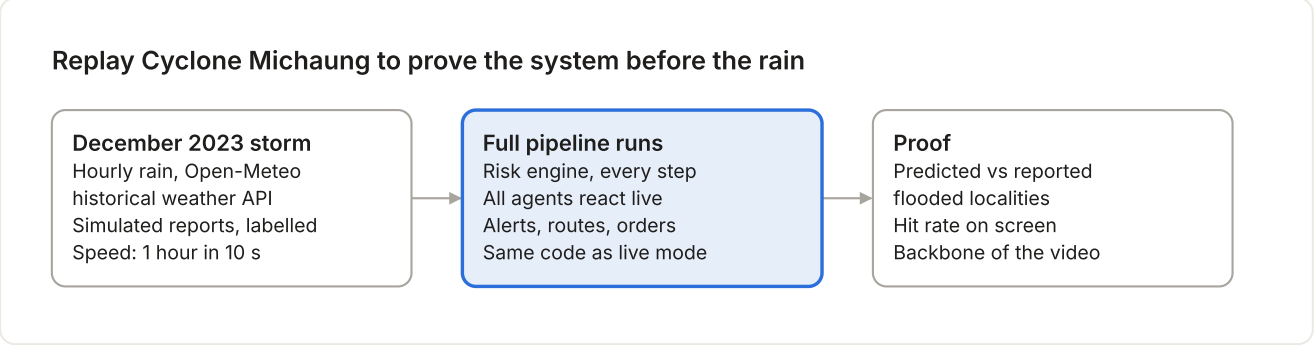


- **SOS form or bot command:** location, number of people, flags for elderly, children or a medical need, and a photo of the water level.
- **Ranking:** rising risk at that location from F2, vulnerability flags, and time waiting. Duplicate calls from the same building are merged.
- **At-risk registry:** residents who would need help to evacuate (elderly living alone, wheelchair users) can opt in before the storm. Only officers and the Triage agent can read it, enforced by Cedar.
- **Shelters:** nearest relief camp with remaining capacity, entered by officers.

Priority: stretch. If time is short, show one ranked SOS card in the console and describe the rest in the video.

F9 · Storm Replay and validation (new)

You can't wait for rain this weekend, so the system replays Cyclone Michaung (December 2023) hour by hour through the real pipeline. It is both the demo and the evidence that the model works.

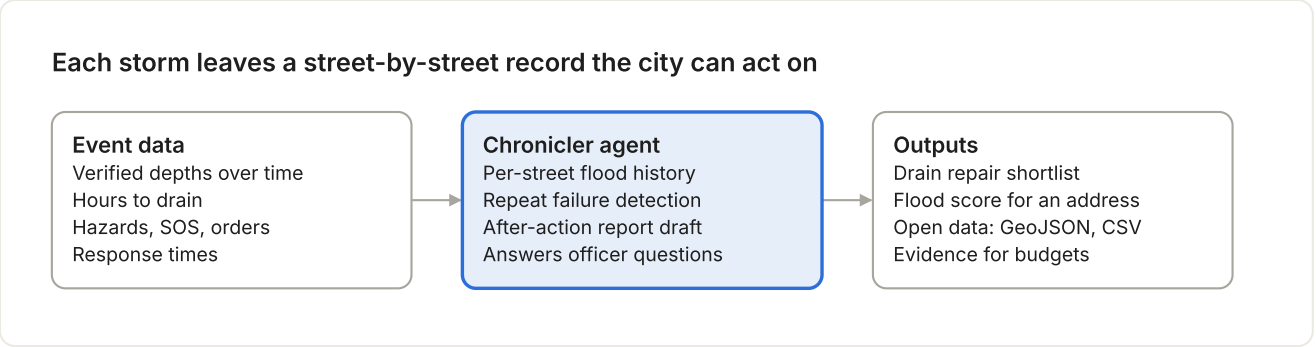


- **Rain:** pull hourly precipitation for the wards from Open-Meteo's historical weather API for the days of the storm and feed it to the risk engine on a fast clock.
- **Reports:** your team's simulated citizen reports, clearly labelled as simulated on screen. Don't use photos scraped from news sites.
- **Validation:** before filming, list the localities news coverage reported as flooded during that storm, then show predicted versus reported flooding and the hit rate. Be honest if it misses some.
- **One codebase:** replay just swaps the clock and the rain source, so everything shown is the same code that runs live.

Priority: must ship. Without it the demo has no rain.

F10 · Flood ledger and after-action reports

Every storm leaves a per-street record: how often it flooded, how deep, how long it took to drain, and how fast the response was. This is what turns a monsoon app into a planning tool.

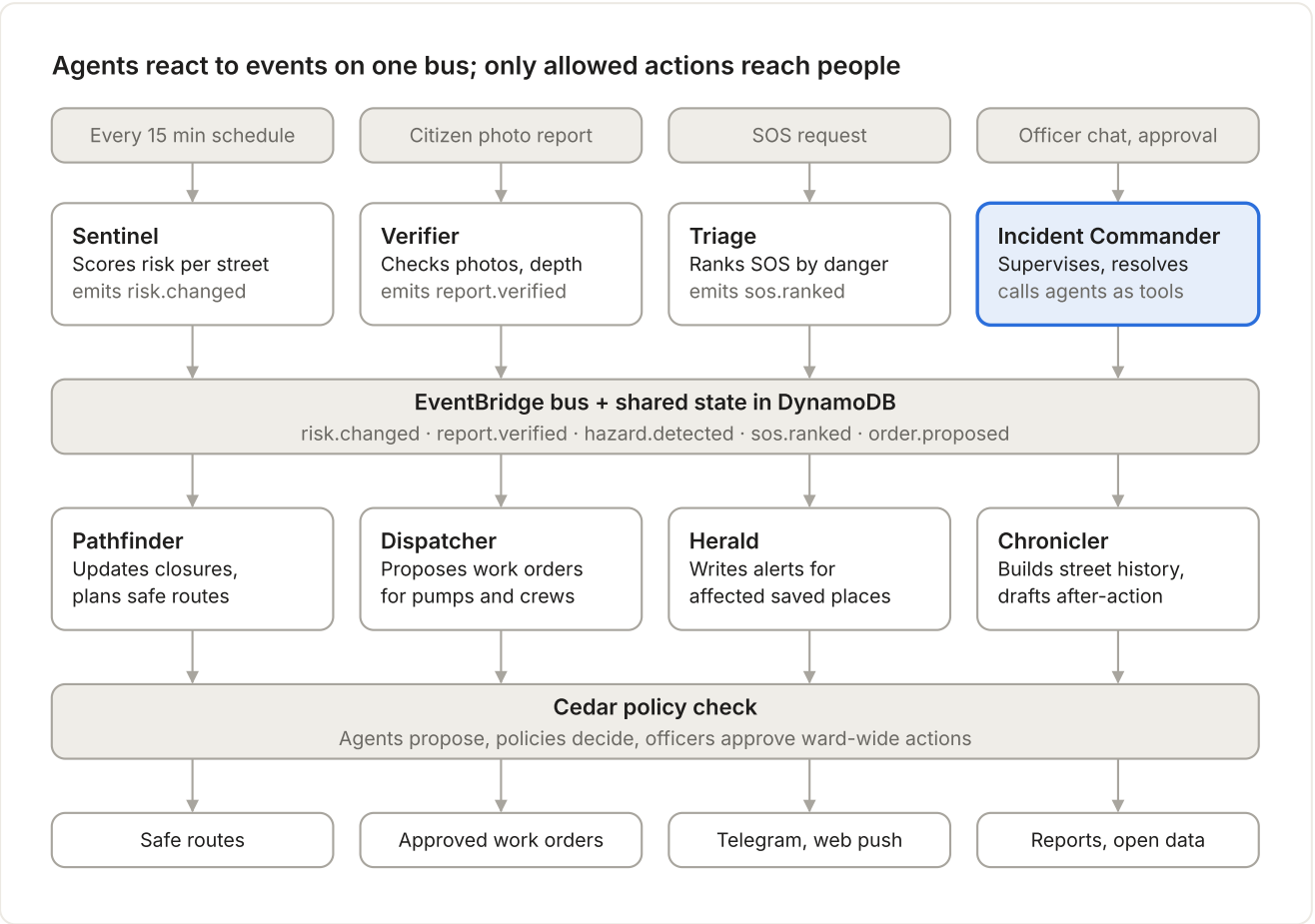


- **Ledger:** per segment, the count of flood events, peak depth, hours to drain, reports, hazards.
- **After-action report:** when the rain ends, the Chronicler drafts a summary of the worst streets, response times from report to dispatch to cleared, and repeat failures. The output is a drain-repair shortlist the officer can forward.
- **Address flood score (new):** anyone can check how a street has behaved across storms before renting or buying.
- **Open data:** GeoJSON and CSV export for researchers and journalists.

Priority: a light version is should-ship, because one after-action report at the end of the Michaung replay is a strong closing shot. The address score is stretch.

Multi-agent system

Eight agents built with the Strands Agents SDK coordinate through an EventBridge bus and shared DynamoDB state. Sensing agents publish events, acting agents react to them, the Incident Commander supervises, and every outward action passes a Cedar policy check.



Agent	Wakes on	Tools	Produces	Model
Sentinel	15-minute schedule	get_forecast, get_tide, score_segments	risk.changed with a plain-language reason	Code scores; small model writes the reason
Verifier	report.created	vision_depth, geo_check, nearby_reports, trust_score	report.verified, hazard.detected	Multimodal model plus rules
Triage	sos.created	risk_at, registry_lookup, nearest_shelter	sos.ranked	Small model plus rules
Pathfinder	risk.changed, report.verified, route requests	build_avoid_zones, route	Closures, routes with reasons	Code plus small model
Dispatcher	risk.changed, hazard.detected, sos.ranked	list_resources, impact_estimate, solve_assignment	order.proposed	Larger model plans; solver assigns
Herald	risk.changed, order.approved	affected_users, compose_alert, send_telegram	Alerts in Tamil and English	Small model inside templates
Chronicler	Storm end, officer questions	query_ledger, write_report	After-action reports, answers	Larger model
Incident Commander	Officer chat, conflicts, escalations	Every other agent as a tool	Briefings, conflict decisions	Larger model

Why several agents instead of one:

- **Different clocks:** a schedule, citizen events and a human chat run at different speeds and shouldn't block each other.
- **Different permissions:** only the Herald sends messages and only the Dispatcher proposes work orders, which Cedar can enforce per agent.
- **Isolation:** each specialist is its own Lambda, triggered by an EventBridge rule, so one failing agent doesn't stall the rest and each can be tested alone.
- **Deterministic core:** risk math, routing and assignment stay in code; agents add judgement, verification and language on top.

The demo moment, a real conflict: one free pump, two needs. The Dispatcher ranks Velachery Main Road first by buildings affected, while Triage flags a family with an elderly member stranded on a side street. The Commander re-ranks, explains the trade-off in the console, and the officer approves. That 20 seconds shows judges actual coordination rather than a chatbot.

Hosting: specialists run as Lambdas; the Commander needs streaming chat, so run it on App Runner, or on Bedrock AgentCore Runtime if your team has used it.

Guardrails and human-in-the-loop

Agents propose, Cedar policies decide, and humans approve anything that is ward-wide or sends a crew. Cedar is an AWS open-source policy language, so it also gives you a second AWS open-source tool alongside Strands.

- Each agent is a Cedar principal; actions include SendAlert, ProposeWorkOrder, DispatchCrew and ReadSOS.
- Cedar denies by default, so an agent can do only what a policy explicitly allows.
- Evaluate policies in Lambda with the `cedarpy` Python binding before any outward action.
- Log every decision (agent, action, allowed or denied, reason) to DynamoDB and show it in the console's activity feed.

```
// Herald may send alerts
permit (principal == Agent::"Herald", action == Action::"SendAlert", resource);

// ...but ward-wide alerts need an officer's approval
forbid (principal, action == Action::"SendAlert", resource)
  when { resource.scope == "ward_wide" }
  unless { context.officer_approved };

// Only the Dispatcher touches work orders, and crews move only after approval
permit (principal == Agent::"Dispatcher",
        action in [Action::"ProposeWorkOrder", Action::"DispatchCrew"], resource);
forbid (principal, action == Action::"DispatchCrew", resource)
  unless { context.officer_approved };

// SOS personal data: the Triage agent and officers only
permit (principal == Agent::"Triage", action == Action::"ReadSOS", resource);
permit (principal in Role::"Officer", action == Action::"ReadSOS", resource);
```

Demo moment: the Herald tries a ward-wide alert, Cedar denies it, it appears in the console as "awaiting approval," the officer approves, and it goes out. Ten seconds that show responsible AI instead of just claiming it.

AWS stack and data sources

The project is deployed on AWS and uses two AWS open-source tools (Strands and Cedar), so it clears the prize-eligibility rule twice over. Deploy the submitted version on your own Free Tier account, not the 8-hour Builder Center sandbox, so the URL still works when judges open it.

Layer	Choice	Used by
Front end	React + MapLibre GL on Amplify Hosting	Citizen app, ward console
API	API Gateway + Lambda (Python)	Reports, routes, SOS, approvals
Agents	Strands Agents SDK; Lambdas, Commander on App Runner	All eight agents
Models	Amazon Bedrock (multimodal model for photos)	Verifier, Commander, Herald, Chronicler
Events	EventBridge bus + EventBridge Scheduler	Agent coordination, 15-minute scoring
Data	DynamoDB (segments, reports, hazards, resources, ledger); S3 (photos, GeoJSON)	Everything
Auth	Cognito for officers; Telegram identity for citizens	Console, bot
Policy	Cedar via cedarpy	Every outward action

Layer	Choice	Used by
Ops	CloudWatch logs and dashboards	Debugging, the agent feed

Data source	What it gives	Notes
Copernicus DEM GLO-30 (AWS Open Data registry)	Elevation for F1	Download once for the ward box
OpenStreetMap via Overpass	Roads, canals, buildings, hospitals	Building counts are the impact proxy
Open-Meteo forecast API	Hourly rain forecast	Free, no key
Open-Meteo historical weather API	Michaung rainfall for Storm Replay	Free, no key
Tide table for Chennai	High-tide windows	Precompute for the weekend
Officer input	Lake releases, shelters, pumps and crews	Entered in the console

Cost: Bedrock calls are the main spend. Use a small model for high-volume steps, cache vision results per photo, and stay inside the Free Tier credits.

What ships, who builds it, and when

Ship eight features and five agents end to end; everything else is shown as roadmap in the video. The submission deadline hour isn't posted yet, so plan to submit by mid-afternoon Sunday.

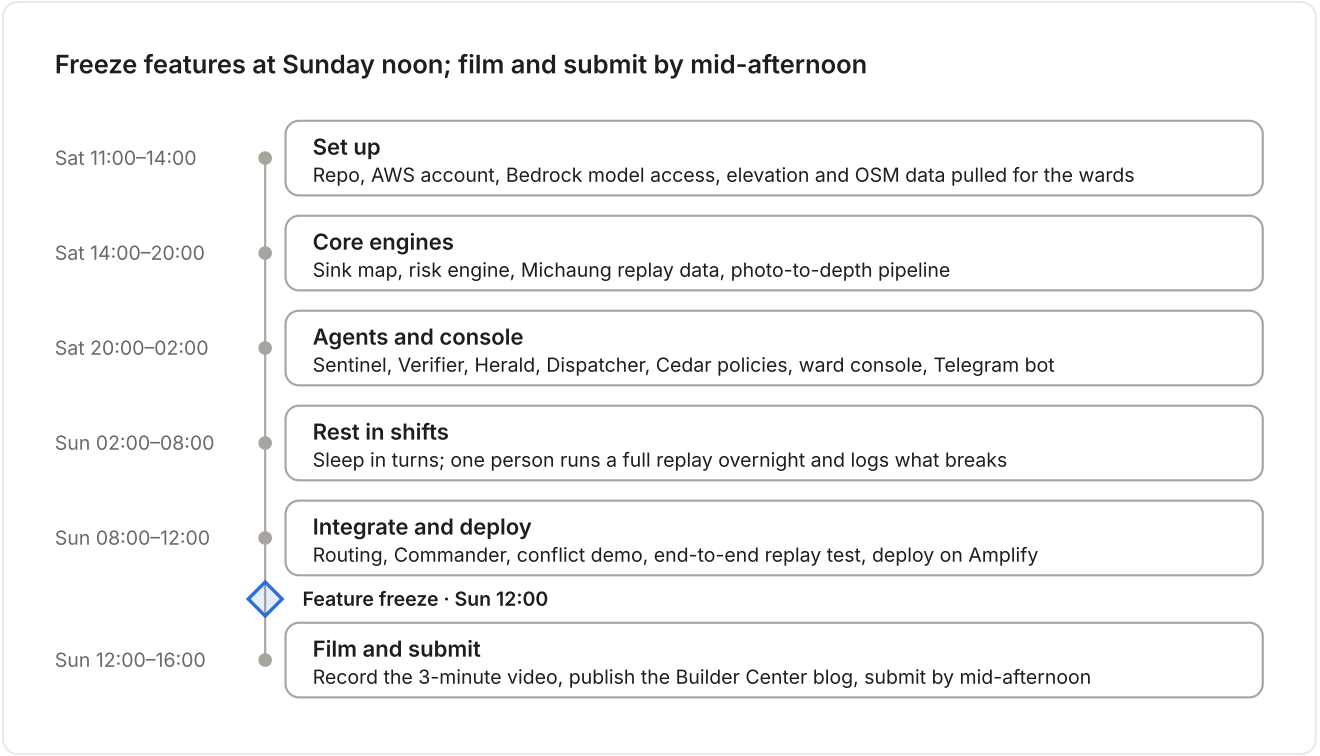
Feature	Priority	Demo moment
F1 Terrain sink map	Must	Low spots shaded on the ward map
F2 Live risk engine	Must	Risk climbing as replay rain rises
F3 Citizen reports	Must	Photo becomes "knee-deep" in seconds
F4 Hazard layer	Must (pin), should (queues)	Open-manhole pin, route avoids it
F5 Smart alerts	Must	Tamil alert lands on a phone
F6 Safe routing	Must	Two-wheeler detour with its reason
F7 Ward console	Must	Approve a dispatch; agent feed scrolling
F9 Storm Replay	Must	Michaung predicted vs reported
F10 Flood ledger	Should (light)	After-action report as the closing shot
F8 SOS triage	Stretch	One ranked SOS card

Agents: Sentinel, Verifier, Herald, Dispatcher and a basic Commander are must-ship. Pathfinder can start as plain routing code and become an agent if time allows. Triage and the full Chronicler are stretch. Cedar needs only the two policies used in the demo (ward-wide alert, crew dispatch).

Team lanes (for four people; merge lanes if you are fewer):

1. **Data and engines:** F1, F2, Storm Replay data.
2. **Agents and policy:** Strands agents, EventBridge wiring, Cedar.

3. **Front end:** citizen app, ward console, map.
4. **Integrations and delivery:** Telegram bot, routing, deployment, video, blog.



The one rule that protects the submission: nothing new starts after the Sunday noon freeze.

Three-minute demo video

The video follows one storm from the first raindrop to the after-action report, so judges see citizen, rider and officer views in a single story. There is no live demo, so this is everything they will see.

Time	On screen	What the voiceover says
0:00–0:15	Real waterlogged Chennai street, your own footage	The same streets flood every monsoon, but alerts are city-wide
0:15–0:35	Storm Replay starts; the ward map shifts from low to likely	NIMBUS predicts flooding street by street from terrain, rain and tide
0:35–0:55	A phone receives a Tamil alert to move a bike	Alerts tell people what to do, for the places they care about
0:55–1:20	Photo upload returns knee-deep plus an open manhole; map updates	Every report is verified and teaches the map
1:20–1:35	Two-wheeler route detours with its reason	Routes avoid water deeper than your vehicle can handle
1:35–2:05	Console: agent feed, pump conflict resolved, Cedar holds a ward-wide alert until approved	Agents propose, policies decide, officers approve
2:05–2:25	Predicted vs reported flooding for Michaung; after-action report	State what the replay actually showed, including misses
2:25–2:50	The architecture diagram from this doc	Where AWS fits: Strands, Cedar, Bedrock, EventBridge, Lambda, DynamoDB, Amplify

Time	On screen	What the voiceover says
2:50–3:00	Team, live URL	Built for Chennai; any city with elevation and rain data can run it

Record the screen at 1080p with captions on, run the replay at a fixed speed so takes match, and film the street shot before you need it in case it doesn't rain.

Open decisions to lock today

Settle these in the next hour; each one is marked with the default to use if nobody objects.

- ☐ Team size and who takes each lane (default: the four lanes above).
- ☐ Wards: Velachery alone, or Velachery plus Pallikaranai (default: Velachery alone).
- ☐ Citizen channel: Telegram bot plus web app, or web app only (default: both, Telegram first).
- ☐ Router: OpenRouteService or Amazon Location Service (default: OpenRouteService).
- ☐ Demo storm: Cyclone Michaung replay (default: yes).
- ☐ Name on the submission: NIMBUS alone, or with a subtitle such as "street-level flood intelligence" (default: both).